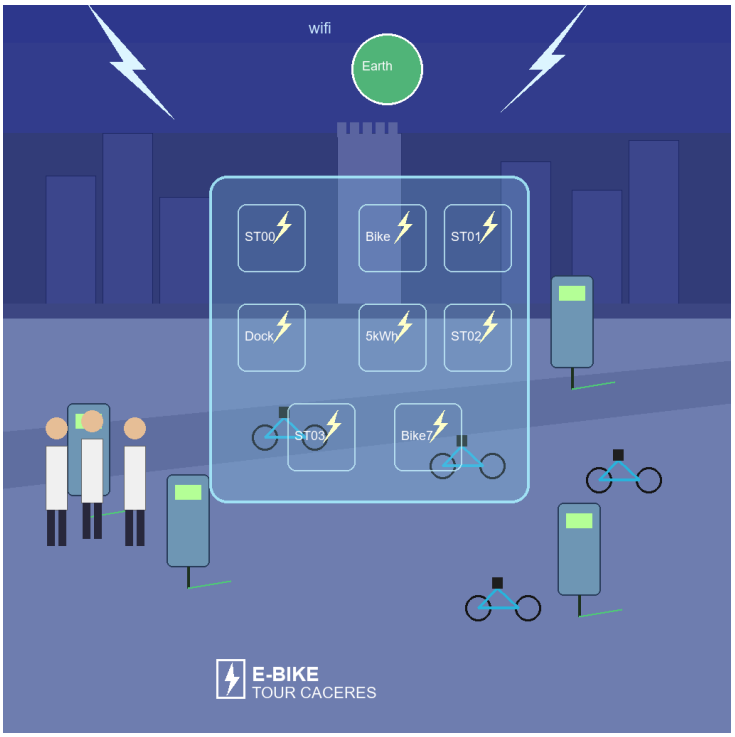
PROYECTO DE LA ASIGNATURA
DESARROLLO DE PROGRAMAS
Curso 2026/2027

Grado en Ingeniería en Informática en:
Ingeniería del Software - Ingeniería de Computadores



E-Bike Tour

Fecha límite: 16 de noviembre

En este documento se especifican las acciones que deben llevarse a cabo para implementar el proyecto de la asignatura Desarrollo de Programas.

1.- Introducción

El objetivo del proyecto de la asignatura es construir un sistema que permita simular una red de estaciones para bicicletas eléctricas en una ciudad. Hemos tomado como referencia ideas de sistemas reales de movilidad sostenible, pero hemos hecho una adaptación ficticia para trabajar los conceptos de Programación Orientada a Objetos impartidos en la asignatura.

El proyecto consistirá en simular cómo una compañía (BikeCompany) gestiona la red de estaciones de carga para bicicletas (BikeStation) para atender solicitudes de uso de bicicletas eléctricas (ElectricBike) que circulan por Cáceres.

En esta primera entrega la compañía está comenzando su negocio, con lo que, por ahora, gestionará los procesos de carga de una manera uniforme, pero en el futuro se prevé que la compañía crezca y posea diferentes tipos de dársena (*docks*), con lo que podrá haber procesos de carga diferentes, así como que existan diferentes tipos de bicicletas, en función de los tipos de

clientes que existan.

En esta primera entrega, la ejecución de la simulación consistirá en lo siguiente:

1. Cada ElectricBike parte de una posición inicial (Location) y se desplazará hasta un destino turístico indicado.
2. Si la bicicleta no tiene carga suficiente para llegar al destino, debería dirigirse antes a una estación de carga para bicicletas.
3. Una vez en la estación, la bici se conectará, consumirá energía simulada y quedará registrada como atendida.
3. Tras ser atendida, la bicicleta repetirá el proceso hasta llegar a su destino final.

La salida de la ejecución de la simulación en esta fase inicial se realizará **por consola**, pero no se descarta que en el futuro (segunda entrega del proyecto) la compañía quiera obtener la salida en un fichero.

2.- Descripción de las clases principales existentes en el proyecto "E-Bike Tour"

2.1- *Location*

La clase **Location** representa una posición concreta en una ciudad. Cada posición tendrá una coordenada ***x*** y una coordenada ***y*** para una posible representación gráfica (en forma de matriz bidimensional).

Requisitos de la clase **Location**:

- cada objeto de la clase **Location** se caracteriza por tener, al menos, los siguientes campos (**x**, **y**):
  - **x**, posición de la **fila** dentro de la matriz. En esta primera entrega sus valores mínimo y máximo serán 0 y 19 respectivamente (para una matriz de 20x20).
  - **y**, posición de la **columna** dentro de la matriz. En esta primera entrega sus valores mínimo y máximo serán 0 y 19 respectivamente (para una matriz de 20x20).

fil/col x / y	0	1	2	3	4	5	6
0	p0						
1		↘↘	→	→	→	p1	
2			↘				DP2
3			p5	↘	p4		
4	DP1	↙					
5	↙		p3	←	←	↖	
6	p6						p2

Figura 1. Ejemplo de la representación gráfica de una ciudad de 7x7.

Funcionalidad de la clase **Location**:
La clase **Location** ya cuenta con parte de su implementación realizada que se recomienda revisar y **entender**. Además, cada grupo debe implementar:

- Método capaz de proporcionar la distancia (método "***distance***") entre la posición actual (valores **x** e **y** del objeto Location) y una posición que se reciba por parámetro. Esta distancia será la distancia en línea recta entre la posición actual y la posición que se recibe por parámetro (es decir, el máximo valor absoluto de la diferencia entre las coordenadas **x** y las coordenadas **y** de ambas posiciones).
  Ejemplos:
  - entre las posiciones p0 (0,0) y p1 (1,5) la distancia será 5: (el máximo valor absoluto entre los valores 1 (1-0) y 5 (5-0)
  - entre las posiciones p2 (6,6) y p3 (5,2) la distancia será 4: (el máximo valor absoluto entre los valores -1 (5-6) y -4 (2-6).
- Método capaz de obtener la siguiente posición (método "***nextLocation***") por la que debe pasar un bicicleta en su camino a una posición destino, para ello en cada paso:
  a. Si la coordenada **x** y la coordenada **y** de la posición destino no es igual a la coordenada **x** ni a la coordenada **y** de la posición actual de la bicicleta, se suma y/o se resta 1 a cada una de las coordenadas de la posición actual para estar un paso más cerca de la posición destino en ambos ejes. Pueden verse ejemplos de este caso en:
     - el primer paso que se da para ir de la posición p0 (0,0) a la posición p1 (1,5), donde se suma 1 a ambas coordenadas de la posición p0 (0,0), **es 1,1**.
     - el primer paso que se da para ir de la posición p2 (6,6) a la posición p3 (5,2), donde se resta 1 a ambas coordenadas de la posición p2 (6,6), **es 5,5**.
     - el primer y segundo paso que se da para ir de la posición p5 (3,2) a la posición p6 (6,0), donde se suma 1 a la posición **x** y se resta 1 a la posición **y** de las coordenadas de la posición p5 (3,2), **son 4,1 y 5,0**.
  b. Si una de las dos coordenadas de la posición actual de la bicicleta es igual a esa misma coordenada de la posición destino, se suma o se resta 1 a la coordenada de la posición actual que no es igual a la de la posición destino. Pueden verse ejemplos de este caso en:
     - el segundo, tercer y cuarto paso que se da para ir de la posición p0 (0,0) a la posición p1 (1,5), donde se suma 1 solo a la coordenada **y, son 1,2, 1,3 y 1,4**.
     - el segundo, tercer y cuarto paso que se da para ir de la posición p2 (6,6) a la posición p3 (5,2), donde se resta 1 solo a la coordenada **y, son 5,4 y 5,3**.
     - el tercer paso que se da para ir de la posición p5 (3,2) a la posición p6 (6,0), donde se suma 1 solo a la coordenada **x, es 6,0**.

  Ejemplos:
  - para llegar desde la posición p0 (0,0) a la posición p1 (1,5) (camino color amarillo) las siguientes posiciones que se van obteniendo en cada llamada a *nextLocation* serán las siguientes: (1,1) (1,2) (1,3) (1,4) y (1,5)
  - para llegar desde la posición p2 (6,6) a la posición p3 (5,2) (camino color rojo) las siguientes posiciones que se van obteniendo en cada llamada a *nextLocation* serán las siguientes: (5,5), (5,4) y (5,3) y (5,2)
  - para llegar desde la posición p0 (0,0) a la posición p4 (3,4) (camino color azul) las siguientes posiciones que se van obteniendo en cada llamada a *nextLocation* serán las siguientes: (1,1), (2,2), (3,3) y (3,4)

2.2- BikeStation

La clase **BikeStation** representa una estación de bicicletas eléctricas. Es aquí donde las bicicletas deben ir si desean recargar su energía. Podrá existir más de 1 estación de bicicletas en la misma ciudad y éstas serán gestionadas por la compañía **E-Bike Tour**.

Requisitos de la clase **BikeStation**:

- cada objeto de la clase **BikeStation** se caracteriza por tener, al menos, los siguientes campos:
    - **idStation,** identificador único de la estación. Está compuesto por 4 caracteres, 2 letras que indican la ciudad (e.g. CC) y a continuación dos dígitos. Un ejemplo sería la estación CC01 (estación 01 de la ciudad de Cáceres).
    - **location**, localización en la que está la estación de bicicletas (será del tipo **Location** y en esta primera entrega serán indicadas en la demo que será entregada por los profesores de la asignatura),
    - **docks** conjunto de objetos de tipo **Dock** (dársena) que están en la estación de bicicletas.
        - esta estructura se mantendrá ordenada de acuerdo a un criterio de comparación de **docks**, que, en la Entrega Inicial será decreciente por su velocidad de carga. En caso de empate, de manera creciente por su tarifa de carga (céntimos de € / wh) y si hubiera otro empate, en orden creciente por su identificador.

Funcionalidad de la clase **BikeStation**:
La clase **BikeStation** debe implementarse con la siguiente funcionalidad como mínimo (cada grupo añadirá, si es el caso, la funcionalidad que crea necesaria):

- Método que devuelve la localización donde está la estación de bicicletas.
- Método que devuelve la colección **docks**.
- Método que permite añadir un nuevo dock de tipo **Dock** a la colección **docks.**
- Método que permite devolver un objeto de tipo **Dock** de la colección **docks.**

2.3- Dock

La clase **Dock** representa un dock (dársena para cargar) de bicicleta eléctrica que estará situado en alguna de las estaciones de bicicletas. Puede haber más de un dock en la misma estación de bicicletas y éstos podrán tener diferentes características y tarifas.

Requisitos de la clase **Dock**:

- cada objeto de la clase **Dock** se caracteriza por tener, al menos, los siguientes campos:
    - **idDock,** identificador único del dock. Está compuesto por 7 caracteres que contendrán el id de la estación de bicicletas donde se encuentre este dock, seguido del carácter "*_" y a continuación dos dígitos. Un ejemplo sería el dock* CC01_02 (dock 02 de la estación 01 de la ciudad de Cáceres).
    - **chargingSpeed**, velocidad de carga a la que puede llegar el dock en wh.
    - **chargingFee,** precio en € / wh de la carga en el dock.
    - **bikesRecharged,** colección con las bicicletas eléctricas que han realizado alguna carga en el dock.
    - **amountCollected**, cantidad en € recaudada por este dock. Será calculada como el producto de los wh cargados en el dock por la tarifa de carga.
    - **free**, valor que indica si el dock está ocupado o libre.

Funcionalidad de la clase **Dock**:
La clase **Dock** debe implementarse con la siguiente funcionalidad como mínimo (cada grupo añadirá, si es el caso, la funcionalidad que crea necesaria):

- Método que devuelve la velocidad de carga del dock.
- Método que devuelve la tarifa de carga del dock.
- Método que permite añadir un nuevo objeto bicicleta a la colección de bicicletas cargadas**.**
- Método que permite actualizar la cantidad recaudada por el dock.
- Método que permita realizar la recarga de una bicicleta eléctrica.

2.4- ElectricBike

La clase **ElectricBike** representa a los objetos de tipo bicicleta eléctrica, que pueden hacer uso de las estaciones de bicicletas que posee la empresa **E-Bike Tour.** En concreto, estas bicicletas podrán hacer uso de alguno de los docks (**Dock**) que las estaciones de bicicletas (**BikeStation**) ponen a su disposición.

Requisitos de la clase **ElectricBike**:

- cada objeto de la clase **ElectricBike** se caracteriza por tener, al menos, los siguientes campos:
  - **idBike**, código de identificación de la bicicleta (cadena compuesta por 6 caracteres).
  - **company,** objeto de tipo **E-Bike Tour** que representa la empresa de docks a la que está suscrita la bicicleta recargable.
  - **location** objeto de tipo **Location** que representa la posición de la ciudad en la que está inicialmente el **ElectricBike**.
  - **targetLocation** objeto de tipo **Location** que representa la posición destino a la que tiene que desplazarse la **ElectricBike.**
  - **idleCount** contador de tipo entero que representa el tiempo (número de pasos o turnos) que la bicicleta está parada sin desplazarse.
  - **batteryCapacity** valor entero que indica los wh de capacidad de la batería (por ejemplo, 60 wh)**.**
  - **batteryLevel** valor entero que indica los wh que tiene actualmente la batería. Este valor será un valor entre 0 y la capacidad total de la batería.
  - **wsCharged** ws que se han cargado en total en la bicicleta.
  - **chargesCount**, indica el número de recargas que la bicicleta ha realizado.
  - **chargestCost**, indica el coste total de las recargas que ha realizado la bicicleta. El coste de cada recarga se calcula como los wh cargados por el coste de la recarga (tarifa del dock).
  - **docks**, colección con las dársenas (tipo Dock) en las que ha realizado alguna carga.

Funcionalidad de la clase **ElectricBike**:
La clase **ElectricBike** ya cuenta con parte de su implementación realizada que se recomienda revisar y entender. Además, cada grupo debe implementar:

- Método capaz de comprobar e informar si una **ElectricBike** tiene energía suficiente para llegar a su destino o no (“*enoughBattery*”). Esta funcionalidad se calculará teniendo en cuenta la distancia de la bicicleta a su targetLocation, la carga que tenga en ese momento y que la bicicleta consume 5 wh por cada paso que da al moverse por la ciudad.
- Método que calcula el destino al que la bicicleta debe desplazarse en función del

desplazamiento a realizar y el nivel de batería actual ("***calculateRoute***"). Este método se comportará del siguiente modo:

a. calculará la distancia que hay desde su posición hasta su targetLocation:
    - si tiene energía suficiente para llegar a su targetLocation, ese será su destino definitivo.
    - si no tiene energía suficiente, comprobará cuál es el dock que más le conviene para llegar a su destino. Este dock será aquel cuya distancia a la posición actual, sumada a la distancia desde el dock a la targetLocation sea menor, siempre que la bicicleta tenga energía suficiente para llegar desde su posición actual a la posición del dock. En caso de empate, se puede seleccionar el primer dock analizado.

- Método que realice la recarga de la bicicleta una vez llegue al punto de recarga ("***recharge***"). Este método debe realizar todas las operaciones que la recarga implica:
    - seleccionar el primer dock libre que haya en la **BikeStation** y se cargará la bicicleta suministrando los wh que fueran necesarios hasta completar la batería de la bicicleta (capacidad de la batería - carga actual en wh);
    - almacenar el dock en el que ha cargado en la colección **docks**;
    - actualizar el coste, wh y veces que la bicicleta ha recargado; actualizar el nivel de la batería de la bicicleta (se cargará hasta el máximo de capacidad), actualizar la lista de bicicletas recargados en el dock, y finalmente, recalcular la ruta de la bicicleta para llegar al destino final (el proceso será el mismo que el explicado en el punto anterior).
- Debe poder calcular la distancia desde su posición hasta la posición a la que tiene que desplazarse (método "***distanceToTargetLocation***"), para ello, utilizará el método "*distance*" implementado en la clase **Location**.
- Debe poder realizar una acción en la simulación (método "***act***"), esta acción está compuesta por los siguientes pasos:
    a. debe obtener y moverse a la siguiente posición que le corresponda en su camino (en función de su destino, estación de bicicletas o destino final):
        - si ya está en su destino:
            - incrementará el contador idleCount.
            - debe permanecer en esa posición sin moverse hasta el final de la simulación.
        - si la siguiente posición coincide con la posición **targetLocation**:
            - registrará que ya ha llegado a su destino.
        - si la siguiente posición coincide con la posición de la estación de bicicletas a la que se dirigía deberá llevar a cabo el proceso de carga comentado en el método "***recharge***", es decir:
            - llevará a cabo el proceso de carga comentado anteriormente en el método "***recharge***".
            - si no hubiera un dock que pueda ser seleccionado, debe permanecer en esa posición sin moverse hasta el final de la simulación.

- Se debe poder mostrar la información inicial y final de un objeto **ElectricBike** siguiendo las plantillas de salida proporcionadas (ver Anexo I).

2.5.- BikeCompany

Representa la **Compañía de recarga de bicicletas eléctricas o recargables** que controla las estaciones de bicicletas y los docks en los que las bicicletas suscritas podrán cargar.

Requisitos de la clase **BikeCompany**:

- cada objeto de tipo **BikeCompany** se caracteriza por tener, al menos, los siguientes campos:
    - **name** identificativo de la compañía de recarga de bicicletas.
    - **subscribedBikes** conjunto de objetos de tipo **ElectricBike** que están suscritos a la compañía.
    - **stations** colección de estaciones de bicicletas que la compañía gestiona.

Funcionalidad de la clase **BikeCompany**:
La clase **BikeCompany** ya cuenta con parte de su implementación realizada que se recomienda revisar y entender. Además, cada grupo debe implementar, entre otros métodos que quieran añadir:

- Método que permite añadir un objeto de tipo **ElectricBike** a la colección **subscribedBikes.**
- Método que permite añadir un objeto de tipo **BikeStation** a la colección **stations.**
- Método que devuelve la estación de bicicletas de una determinada localización.

2.6.- *Demo*

Representa a la clase capaz de gestionar la simulación de cargas y desplazamientos de los **ElectricBike** a través de la ciudad gestionada por la empresa **BikeCompany.**

Requisitos de la clase **Demo**:

- se caracteriza por tener, al menos, los siguientes campos:
    - **company** objeto de tipo **BikeCompany** con el que se realizará la simulación.
    - **bikes**, colección con los objetos de tipo **ElectricBike** que se crearán para moverse por la ciudad, hay que guardarlas en la compañía. En la Entrega Inicial las bicicletas serán únicamente de tipo **ElectricBike**, pero en la siguiente entrega existirán más tipos de bicicletas en la ciudad. Inicialmente deben quedar ordenadas de manera creciente por su **idBike**.
    - **stations**, colección con los objetos de tipo **BikeStation** que se crearán en la simulación, hay que guardarlos en la compañía. Deben quedar ordenados de manera creciente por su identificador **idStation**. Cada station contendrá también una colección de docks que serán creados en esta demo para la simulación.

Funcionalidad de la clase **Demo**:

- Debe ser capaz de resetear la ejecución de la simulación para dejarla en un punto inicial y que pueda volver a ser iniciada con la misma información de partida (método "***reset***"). Para ello, en este método se realizarán las siguientes acciones:
    a. se eliminarán las posibles bicicletas y estaciones de bicicletas existentes previamente.
    b. se crearán los objetos de tipo **ElectricBike** y se almacenarán en la compañía.
    c. se crearán los objetos de tipo **BikeStation** y se insertarán en la compañía.
    d. se crearán los objetos de tipo **Dock** y se insertarán en las estaciones de bicicletas indicadas en el código.
    e. se mostrará la información al inicio de la simulación. Esta información se organizará de la siguiente forma (método "***showInitialInfo***") (ver ANEXO I con el detalle de la salida por pantalla):
        - se mostrará el nombre de la compañía.
        - se mostrarán los objetos **ElectricBike** en orden creciente por su **idBike**.

        - se mostrarán las estaciones de bicicletas en orden creciente por su **idStation**.
        - la información mostrada de cada estación de bicicletas debe mostrar también la información de sus docks ordenados según el mismo criterio con el que están ordenados en las estaciones de bicicletas (primero por velocidad de carga decreciente, después por tarifa creciente y por último por idDock). Es decir, la información de los docks aparecerá intercalada entre las diferentes estaciones de bicicletas.
    f. se ejecutará el método "***run***" que debe llevar a cabo la simulación de la demo. En concreto, en cada turno de la simulación se realizarán las acciones indicadas para los **ElectricBike** (ver apartado **ElectricBike**). Las **ElectricBike** realizarán sus acciones en orden creciente por su idBike.
        - en cada turno se debe mostrar la información resultante de las acciones que ha llevado a cabo cada **ElectricBike, mostrando los datos de cada bicicleta según lo especificado en el ANEXO I**.
- Debe ser capaz de ejecutar la simulación un número concreto de pasos (método "***run***"). Se simularán 50 pasos en la simulación pero este valor se podrá modificar para hacer pruebas con diferente número de pasos.
- Debe ser capaz de ejecutar un paso concreto de la ejecución (método "***step***"). En cada paso se invocará al método "*act*" de cada bicicleta en el **orden especificado anteriormente** (creciente por su idBike).
- Debe poder mostrar la información al final de la simulación. Esta información se organizará de la siguiente forma (método "***showFinalInfo***"):
    a. se mostrarán los objetos **ElectricBike** en orden creciente del turno en el que han llegado a su destino y en caso de empate, por orden creciente de sus idBike.
        - cada objeto **ElectricBike** mostrará también la colección de **docks** en los que ha cargado (únicamente sus nombres), en el orden inicial de realización de carga.
    b. se mostrarán los objetos **BikeStation** en orden decreciente por el número de bicicletas que se han cargado en los mismos y en caso de empate de manera creciente por su idStation. Para cada BikeStation se mostrará también sus docks y para cada dock, las bicicletas recargadas en el mismo (ver ANEXO I).

3.- Datos iniciales proporcionados

Se proporcionará un proyecto en BlueJ llamado "**BikeInitial**" con las clases explicadas en este documento. Algunas clases y métodos están incompletas para que los estudiantes puedan ir completando el código del proyecto. Se aconseja seguir el orden de implementación según el orden de explicación de clases del apartado "*2.- Descripción de las clases principales existentes en el proyecto "E-Bike Tour"*" de este documento (es decir: *Location*, *BikeStation*, etc.).
Se podrán añadir nuevos elementos al proyecto (clases, métodos, etc.) siempre que esté justificado su uso según la funcionalidad definida.

Para poder ejecutar la simulación con los mismos datos de partida se proporciona un fichero de tipo Demo que contiene una constante que puede tomar 3 valores diferentes de un tipo enumerado: SIMPLE, MEDIUM, ADVANCED. En función de ese valor, se ejecutará una demo con 2, 5 u 8 bicicletas, respectivamente. En todos los casos, el número de estaciones de bicicletas serán 4 y cada una de ellas tendrá 4 docks distintos. Para cada una de esas demos, se entrega un fichero de texto que contiene la salida por pantalla que se debería obtener en la ejecución de

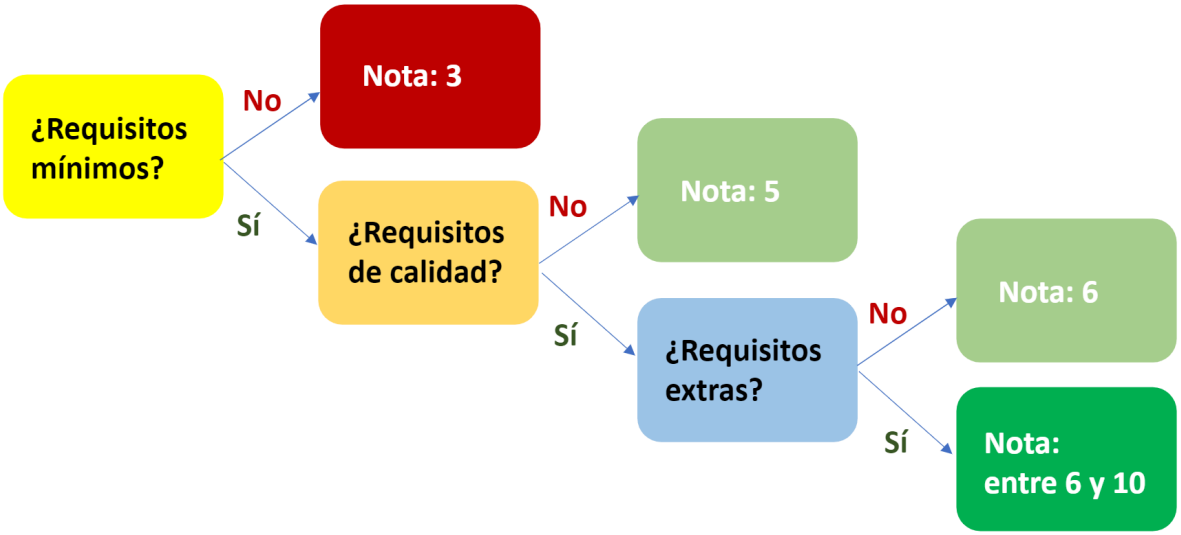
dicha demo. Sin embargo, es recomendable que los estudiantes prueben su proyecto con escenarios diferentes, no contemplados en este enunciado.

La carpeta con el proyecto **BikeInitial** está en la carpeta del proyecto del Campus Virtual (CVUEx).

4.- Requisitos y Evaluación

En este apartado se definen los **requisitos mínimos,** los **requisitos de calidad** y los **requisitos extras de la entrega inicial del proyecto.**

La **forma** en que el **cumplimiento de estos requisitos afecta a la nota final** se muestra en la siguiente figura:



a. Requisitos mínimos

5. **Todas las clases deben incluir los constructores, métodos set y get de cada campo y métodos auxiliares que se consideren necesarios** (por ejemplo métodos **toString, equals, hashCode**, etc..) para que su funcionalidad sea correcta. Además, deben definir correctamente su ámbito de uso con los **modificadores de acceso correspondientes**. Las clases no deben contener funcionalidad que no sea requerida para implementar los requisitos funcionales del sistema, es decir, no deben incluir funcionalidad y/o código que no sean necesarios.

6. El proyecto **debe proporcionar la funcionalidad detallada** en este documento. Para ello, **se deberá poder comprobar el correcto funcionamiento** utilizando una clase llamada "***EBDemo.java***" desarrollada por cada grupo que leerá el conjunto de datos de inicialización que se utilizará en cada caso para realizar la simulación e invocará el/los método/s necesarios para llevar a cabo la simulación. **La funcionalidad proporcionada debe asegurar que**:
    a. El sistema funciona siempre dentro de unos parámetros de tiempo y uso de recursos adecuados.
    b. El sistema controla todos los posibles errores de funcionamiento, avisando al

usuario correctamente de su causa.

- La ejecución de la simulación del proyecto generará una salida **por pantalla**. Esta salida **deberá seguir el formato especificado en la plantilla de salida disponible en la carpeta con el enunciado del proyecto**.
    - Se puede tomar como referencia de salida de ejemplo con esta plantilla los 3 ficheros de salida generados para los 3 escenarios de ejecución descritos anteriormente (SIMPLE, MEDIUM, ADVANCED).
- El proyecto entregado por cada grupo **será capaz de adaptarse a cualquier conjunto de datos iniciales y generar la salida adecuada**.
    - En el código del proyecto al menos deberán entregarse el fichero de tipo Demo (proporcionado en el proyecto) así como la salida generada por el proyecto utilizando cada uno de los escenarios de de datos iniciales (SIMPLE, MEDIUM, ADVANCED).

- El proyecto debe ejecutarse desde el principio hasta el final **sin necesidad de solicitar la intervención del usuario**.

- **Aplicación correcta de los conceptos clase, encapsulación y composición**:
    - Se definen correctamente las clases necesarias para el proyecto, especificando adecuadamente su interfaz funcional pública.
    - Todas las clases aplican correctamente el principio de encapsulación sin exponer en su interfaz funcional pública detalles de implementación de sus datos.
    - Se utiliza adecuadamente la composición en la definición de las clases del proyecto.

- En el proyecto se habrá hecho un **uso correcto de Collections:** (mínimo **List**)
    - Se utilizarán correctamente, junto con los métodos que proporcionan, **al menos una List**, utilizándola de forma apropiada en cada caso, de acuerdo a sus características.

---

c. Requisitos de calidad

- **Estilo de código adecuado. El código entregado cumple las especificaciones del estilo de código vistas a lo largo de la asignatura**, entre las que por ejemplo se encuentran:
    - Correcta indentación de todo el código
    - Uso de un único estilo en los nombres de clases, variables y métodos así como en los comentarios, nombres de ficheros, declaraciones de funciones, bucles, especificación de condiciones e inicialización de variables.
    - Utilizar de forma adecuada las distintas estructuras de iteración. En este sentido:
        - solo se utilizan bucles for para iteración definida
        - bucles while para iteración indefinida

- **Documentación interna y externa adecuada. El código entregado usa de forma correcta la funcionalidad JavaDoc y sus anotaciones para proporcionar:**
    - un resumen de lo que realiza cada clase

- la explicación de todos sus métodos (y sus entradas y salidas) correspondientes (@param, @return, etc.)

- **Formato de entrega adecuado:**
  - El proyecto entregado seguirá las especificaciones del formato de entrega detalladas en el apartado 5 de este enunciado.

---

d. Requisitos extras

Como aportaciones adicionales a la entrega se pueden realizar las siguientes:

- **Uso correcto de Iterator** (**0,5 puntos**):
  - Se utilizará correctamente, en la funcionalidad apropiada a sus características, junto con los métodos que proporciona, **al menos dos veces**.

Como aportación adicional a la entrega se pueden realizar las siguientes:

- **Uso correcto de la funcionalidad ofrecida por las Colecciones y los Comparadores** (hasta **1 punto**):
  - Se utilizará en cada momento el método más adecuado de los ofrecidos por las Collections para operar sobre los campos de tipo colección de cada clase.
  - Se entregarán, **al menos**, los siguientes comparadores implementados de forma correcta (o se implementará correctamente en la parte correcta su alternativa usando Lambdas y Streams):
    - Uno que compare los *ElectricBike* por su indicador en orden creciente.
    - Uno que compare los *ElectricBike* por el turno en el que han llegado a su destino
    - Uno que compare los *BikeStation* por el número de bicicletas cargadas en docks de la estación en orden decreciente y en caso de empate por su id en orden creciente.
    - Uno que compare los Dock por su velocidad de carga en orden decreciente, en caso de empate por su tarifa en orden creciente y si hay empate por el id en orden creciente.

- **Pruebas unitarias del proyecto con JUnit** (hasta **2 puntos**):
  - Se ha elaborado un conjunto completo de pruebas de los **métodos** de las **clases** (y de los **métodos específicos** de sus **posibles subclases**). **Como mínimo** se incluirán pruebas para **las clases y métodos siguientes:**
    - Location:
      - **testDistance** probará el método "*distance*" de la clase Location (este método se aporta implementado en el proyecto pero puede ser modificado si se desea).
      - **testAdjacentLocations** probará el método "*nextLocation*" de la clase Location, comprobando si dos posiciones son o no adyacentes.
    - Dock:
      - **testRecharge** probará el método "*recharge*" que devuelve el coste de la recarga realizada e incluye la bicicleta recargada en el listado mantenido por la clase Dock.

        - ElectricBike:
            - **testCreation** probará la creación correcta de objetos de tipo ElectricBike, comprobando que la inicialización de campos es correcta.
            - **testEnoughBattery** probará que el cálculo de la energía para ir hasta un destino dado es correcto.
            - **testCalculateRoute** probará que dado un destino y una posición inicial, la bicicleta calcula correctamente la ruta a seguir (si debe pasar o no por un punto de recarga y qué punto es).
            - **testRecharge** comprueba que la recarga de la bicicleta se realiza correctamente.
        - BikeStation:
            - **testAddDock** comprueba que la operación para añadir un nuevo dock a la estación de bicicletas funciona correctamente.
    - Es decir, estas pruebas contendrán **un método de test** con sus correspondientes **aserción/es para cada método público** que **devuelva** algún valor (**excepto** los métodos **get** de los campos de la clase) o **realice algún cambio en el estado de los objetos** (**excepto** de los métodos **set** de los campos de la clase) de la clase a probar.
    - Además, debe incluir de forma obligatoria **la creación de objeto/s necesario/s para las pruebas mediante el uso de *Fixtures***.

- **Uso de la funcionalidad de Git proporcionada por BlueJ** (hasta **1,5 puntos**):
    - Se valorará que se haya hecho un **uso adecuado y continuado** a lo largo del desarrollo del proyecto (al menos desde 2 semanas antes de la fecha de la entrega).
    - Se valorará de forma **individual** a cada miembro del grupo de acuerdo a los commits significativos realizados por cada miembro.

5.- Formato de la entrega

El código de la entrega contendrá **exclusivamente el código necesario para satisfacer las especificaciones** de la misma. Para realizar esta entrega se habilitará una **tarea en el aula virtual**.

Requisitos sobre la entrega:

- Además de tener en cuenta todas las consideraciones detalladas en el apartado "Requisitos mínimos" de este enunciado, los estudiantes deben también tener en cuenta:
    1. El proyecto entregado **sólo contendrá la carpeta del proyecto en BlueJ, eliminando los ficheros ".class" y ".ctxt" y otras carpetas (como "doc") del proyecto BlueJ. Es decir, una carpeta cuyo contenido será exclusivamente**:
        a. Las **clases Java** que implementan la funcionalidad del proyecto
        b. El fichero "**package**" del proyecto BlueJ
        c. Un fichero "**Resumen.txt**" (o "**Readme.txt**") que especifique:
            i. nombre y apellido de cada miembro del grupo
            ii. nombre de la clase que hay que ejecutar para realizar una simulación
            iii. los extras que en opinión del grupo tiene su proyecto
    2. El nombre de dicha carpeta será:
        a. apellidos1y2DeEstudiante1_apellidos1y2DeEstudiante2_apellidos1y2DeEstudiante3(*)
            i. **(*)** se aplicará el orden alfabético sobre el primer apellido para

decidir el orden de los estudiantes

3. El proyecto entregado debe poder **abrirse y probarse** en el entorno **BlueJ** 5.2.0. o posterior con Java 11 o posterior sin que sea necesario por parte de los profesores realizar ninguna acción adicional para su ejecución y pruebas. Si se ha realizado en otro entorno de desarrollo, el grupo debe asegurarse que se abre y ejecuta correctamente en BlueJ.
4. La carpeta del proyecto deberá entregarse comprimida:
    a. El formato a usar será **exclusivamente** el formato **.zip.**
    b. El nombre de este fichero.zip tendrá el mismo nombre que la carpeta del proyecto

- **El incumplimiento de cada uno de los cuatro requisitos de la entrega restará 1 punto de la nota final hasta un máximo de 4 puntos si se incumplen los cuatro requisitos.**

6.- Fecha de la entrega

Cada grupo de estudiantes deberá realizar la **entrega del proyecto antes de las 13:30 horas del 16 de noviembre**.
Es suficiente que el fichero .zip de la entrega sea subido por UN SOLO MIEMBRO DEL GRUPO, siempre que cumpla todos los requisitos sobre el formato de la entrega.

Anexo I. Salida por pantalla esperada de la simulación.

Para comprobar que los resultados del proyecto son correctos, cada grupo deberá implementar una salida por consola acorde al formato que se detalla a continuación.

1. INFORMACIÓN INICIAL.

Inicialmente, se mostrará la información inicial del estado del sistema, que incluye: el nombre de la compañía, el listado de las bicicletas en el sistema, el listado de las estaciones de bicicletas y el listado de docks para cada estación de bicicletas. A continuación se muestra un ejemplo de esta salida y se explica el significado de cada campo en los diferentes tipos de línea:

Ejemplo de salida con información inicial:

```
( Compañía EBBike Cáceres )
(--------------------)
( Electric Bikes )
(--------------------)
(ElectricBike: EBCC0, 15wh, 15wh, 0, 0.0€, 0, 10-13, 10-11, 1-1)
(ElectricBike: EBCC1, 30wh, 30wh, 0, 0.0€, 0, 8-4, 10-5, 19-19)
(--------------------)
( Bike Stations )
(--------------------)
(BikeStation: CC00, 0, 10-5)
(Dock: CC00_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC00_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC00_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC00_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC01, 0, 10-11)
(Dock: CC01_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC01_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC01_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC01_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC02, 0, 14-16)
(Dock: CC02_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC02_002, 60wh, 0.6€, 0, 0.0€)
```

```
(Dock: CC02_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC02_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC03, 0, 8-4)
(Dock: CC03_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC03_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC03_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC03_000, 20wh, 0.2€, 0, 0.0€)
```

A continuación se explica el significado de la información mostrada para cada bicicleta eléctrica con un ejemplo:

```
(ElectricBike: EBCC0, 15wh, 15wh, 0, 0.0€, 0, 10-13, 10-11, 1-1)
```

Esta línea muestra la información de un bicicleta eléctrica con los siguientes datos:

- idBike: EBCC0
- Capacidad de batería: 15wh
- Nivel actual de batería: 15wh
- Número de recargas: 0
- Coste total de recargas: 0.0€
- Número de turnos inactivo (idleCount): 0
- Localización actual: 10-13
- Localización de estación de bicicletas a la que se moverá (este campo es opcional): 10-11
- Localización de su destino final: 1-1

A continuación se explica el significado de la información mostrada para cada estación de bicicletas:

```
(BikeStation: CC00, 0, 10-5)
```

Esta línea muestra la información de una estación de bicicletas con los siguientes datos:

- Identificador: CC00
- Bicicletas recargadas: 0
- Localización: 10-5

A continuación se explica el significado de la información mostrada para cada dock:

```
(Dock: CC00_003, 80wh, 0.8€, 0, 0.0€)
```

Esta línea muestra la información de un dock con los siguientes datos:

- Identificador: CC00_003
- Velocidad de carga: 80wh
- Tarifa de carga: 0.8€
- Bicicletas recargadas: 0
- Cantidad recaudada: 0.0€

2. INFORMACIÓN DE LA SIMULACIÓN.

Una vez mostrada la información inicial, se mostrará la información de cada paso realizado en la simulación. Para ello, por cada paso que una bicicleta eléctrica realice, se mostrará su información una vez realizado el paso. Adicionalmente, cuando la bicicleta realice una recarga o llegue a su destino, se mostrará un mensaje también con información de ese hecho. A continuación se muestra un ejemplo de salida y se explican las líneas mostradas.

```
(------------------)
( Simulation start )
(------------------)
(step: 0 - ElectricBike: EBCC0, 15wh, 10wh, 0, 0.0€, 0, 10-12, 10-11, 1-1)
```

```
(step: 0 - ElectricBike: EBCC1, 30wh, 25wh, 0, 0.0€, 0, 9-5, 10-5, 19-19)
(step: 1 - ElectricBike: EBCC0 recharges: 10wh at dock: CC01_003 with cost: 8.0€ ********)
(step: 1 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 0, 10-11, 1-1)
(step: 1 - ElectricBike: EBCC1 recharges: 10wh at dock: CC00_003 with cost: 8.0€ ********)
(step: 1 - ElectricBike: EBCC1, 30wh, 30wh, 1, 8.0€, 0, 10-5, 10-11, 19-19)
(step: 2 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 1, 10-11, 1-1)
(step: 2 - ElectricBike: EBCC1, 30wh, 25wh, 1, 8.0€, 0, 10-6, 10-11, 19-19)
(step: 3 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 2, 10-11, 1-1)
(step: 3 - ElectricBike: EBCC1, 30wh, 20wh, 1, 8.0€, 0, 10-7, 10-11, 19-19)
(step: 4 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 3, 10-11, 1-1)
(step: 4 - ElectricBike: EBCC1, 30wh, 15wh, 1, 8.0€, 0, 10-8, 10-11, 19-19)
(step: 5 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 4, 10-11, 1-1)
(step: 5 - ElectricBike: EBCC1, 30wh, 10wh, 1, 8.0€, 0, 10-9, 10-11, 19-19)
(step: 6 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 5, 10-11, 1-1)
(step: 6 - ElectricBike: EBCC1, 30wh, 5wh, 1, 8.0€, 0, 10-10, 10-11, 19-19)
(step: 7 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 6, 10-11, 1-1)
(step: 7 - ElectricBike: EBCC1 recharges: 30wh at dock: CC01_003 with cost: 24.0€ ********)
(step: 7 - ElectricBike: EBCC1, 30wh, 30wh, 2, 32.0€, 0, 10-11, 14-16, 19-19)
(step: 8 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 7, 10-11, 1-1)
(step: 8 - ElectricBike: EBCC1, 30wh, 25wh, 2, 32.0€, 0, 11-12, 14-16, 19-19)
(step: 9 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 8, 10-11, 1-1)
(step: 9 - ElectricBike: EBCC1, 30wh, 20wh, 2, 32.0€, 0, 12-13, 14-16, 19-19)
(step: 10 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 9, 10-11, 1-1)
(step: 10 - ElectricBike: EBCC1, 30wh, 15wh, 2, 32.0€, 0, 13-14, 14-16, 19-19)
(step: 11 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 10, 10-11, 1-1)
(step: 11 - ElectricBike: EBCC1, 30wh, 10wh, 2, 32.0€, 0, 14-15, 14-16, 19-19)
(step: 12 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 11, 10-11, 1-1)
(step: 12 - ElectricBike: EBCC1 recharges: 25wh at dock: CC02_003 with cost: 20.0€ ********)
(step: 12 - ElectricBike: EBCC1, 30wh, 30wh, 3, 52.0€, 0, 14-16, 19-19)
(step: 13 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 12, 10-11, 1-1)
(step: 13 - ElectricBike: EBCC1, 30wh, 25wh, 3, 52.0€, 0, 15-17, 19-19)
(step: 14 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 13, 10-11, 1-1)
(step: 14 - ElectricBike: EBCC1, 30wh, 20wh, 3, 52.0€, 0, 16-18, 19-19)
(step: 15 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 14, 10-11, 1-1)
(step: 15 - ElectricBike: EBCC1, 30wh, 15wh, 3, 52.0€, 0, 17-19, 19-19)
(step: 16 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 15, 10-11, 1-1)
(step: 16 - ElectricBike: EBCC1, 30wh, 10wh, 3, 52.0€, 0, 18-19, 19-19)
(step: 17 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 16, 10-11, 1-1)
(step: 17 - ElectricBike: EBCC1 at target destination ********)
(step: 17 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 0, 19-19, 19-19)
(step: 18 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 17, 10-11, 1-1)
(step: 18 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 1, 19-19, 19-19)
(step: 19 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 18, 10-11, 1-1)
(step: 19 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 2, 19-19, 19-19)
(step: 20 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 19, 10-11, 1-1)
(step: 20 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 3, 19-19, 19-19)
(step: 21 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 20, 10-11, 1-1)
(step: 21 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 4, 19-19, 19-19)
(step: 22 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 21, 10-11, 1-1)
(step: 22 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 5, 19-19, 19-19)
(step: 23 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 22, 10-11, 1-1)
(step: 23 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 6, 19-19, 19-19)
(step: 24 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 23, 10-11, 1-1)
(step: 24 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 7, 19-19, 19-19)
(step: 25 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 24, 10-11, 1-1)
(step: 25 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 8, 19-19, 19-19)
(step: 26 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 25, 10-11, 1-1)
(step: 26 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 9, 19-19, 19-19)
(step: 27 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 26, 10-11, 1-1)
(step: 27 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 10, 19-19, 19-19)
(step: 28 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 27, 10-11, 1-1)
(step: 28 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 11, 19-19, 19-19)
(step: 29 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 28, 10-11, 1-1)
(step: 29 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 12, 19-19, 19-19)
(step: 30 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 29, 10-11, 1-1)
(step: 30 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 13, 19-19, 19-19)
(step: 31 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 30, 10-11, 1-1)
```

```
(step: 31 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 14, 19-19, 19-19)
(step: 32 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 31, 10-11, 1-1)
(step: 32 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 15, 19-19, 19-19)
(step: 33 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 32, 10-11, 1-1)
(step: 33 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 16, 19-19, 19-19)
(step: 34 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 33, 10-11, 1-1)
(step: 34 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 17, 19-19, 19-19)
(step: 35 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 34, 10-11, 1-1)
(step: 35 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 18, 19-19, 19-19)
(step: 36 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 35, 10-11, 1-1)
(step: 36 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 19, 19-19, 19-19)
(step: 37 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 36, 10-11, 1-1)
(step: 37 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 20, 19-19, 19-19)
(step: 38 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 37, 10-11, 1-1)
(step: 38 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 21, 19-19, 19-19)
(step: 39 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 38, 10-11, 1-1)
(step: 39 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 22, 19-19, 19-19)
(step: 40 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 39, 10-11, 1-1)
(step: 40 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 23, 19-19, 19-19)
(step: 41 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 40, 10-11, 1-1)
(step: 41 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 24, 19-19, 19-19)
(step: 42 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 41, 10-11, 1-1)
(step: 42 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 25, 19-19, 19-19)
(step: 43 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 42, 10-11, 1-1)
(step: 43 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 26, 19-19, 19-19)
(step: 44 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 43, 10-11, 1-1)
(step: 44 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 27, 19-19, 19-19)
(step: 45 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 44, 10-11, 1-1)
(step: 45 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 28, 19-19, 19-19)
(step: 46 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 45, 10-11, 1-1)
(step: 46 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 29, 19-19, 19-19)
(step: 47 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 46, 10-11, 1-1)
(step: 47 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 30, 19-19, 19-19)
(step: 48 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 47, 10-11, 1-1)
(step: 48 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 31, 19-19, 19-19)
(step: 49 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 48, 10-11, 1-1)
(step: 49 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
```

En cada paso, para cada bicicleta se mostrará su información con el mismo formato explicado en la sección de información inicial. Se muestran a continuación varios ejemplos.

Ejemplo 1:

```
(step: 0 - ElectricBike: EBCC0, 15wh, 10wh, 0, 0.0€, 0, 10-12, 10-11, 1-1)
```

En este caso, se muestra un bicicleta que tras realizar el paso 0 de la simulación (primero) está situado en la posición 10-12, debe ir a la estación de bicicletas situada en la posición 10-11 y su destino final es la posición 1-1.

Ejemplo 2:

```
(step: 12 - ElectricBike: EBCC1 recharges: 25wh at dock: CC02_003 with cost: 20.0€ ********)
(step: 12 - ElectricBike: EBCC1, 30wh, 30wh, 3, 52.0€, 0, 14-16, 19-19)
```

Se muestra que la bicicleta con id EBCC1 ha realizado una recarga en el dock CC02_003 de 25wh con un coste de 20.0€ en el turno 12. A continuación se muestra su estado al final de ese turno, mostrando que está en la posición 14-16 y que su destino final es la posición 19-19 (en este caso, no hay posición intermedia en la que cargar porque tiene energía suficiente para ir al destino final).

Ejemplo 3:

```
(step: 17 - ElectricBike: EBCC1 at target destination ********)
(step: 17 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 0, 19-19, 19-19)
```

Se muestra que la bicicleta con id EBCC1 ha llegado a su destino final en el turno 17. A continuación se muestra su estado al final de ese turno, mostrando que está en la posición 19-19 que es la misma que su destino final. Esta bicicleta no se moverá hasta el final de la simulación, por lo que en los siguientes turnos aparecerá que está siempre en esa misma posición.

3. INFORMACIÓN FINAL.

Una vez terminada la simulación, se mostrará la información del estado final del sistema. A continuación se muestra un ejemplo junto con la descripción de cada parte mostrada.

```
( Final information )
(-------------------)
(-------------------)
( Electric Bikes )
(-------------------)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC00_003)
(Dock: CC01_003)
(Dock: CC02_003)
(ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 48, 10-11, 1-1)
(Dock: CC01_003)
(-------------------)
( Bike Stations )
(-------------------)
(BikeStation: CC01, 2, 10-11)
(Dock: CC01_003, 80wh, 0.8€, 2, 32.0€)
(ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 48, 10-11, 1-1)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC01_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC01_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC01_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC00, 1, 10-5)
(Dock: CC00_003, 80wh, 0.8€, 1, 8.0€)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC00_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC00_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC00_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC02, 1, 14-16)
(Dock: CC02_003, 80wh, 0.8€, 1, 20.0€)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC02_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC02_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC02_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC03, 0, 8-4)
(Dock: CC03_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC03_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC03_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC03_000, 20wh, 0.2€, 0, 0.0€)
```

Como puede verse en el ejemplo mostrado, en primer lugar se muestra la información de las bicicletas del sistema ordenadas de manera creciente por el turno en el que han llegado a su destino (y en caso de empate, por su id creciente). Si un bicicleta no ha llegado a su destino, se interpretará que es el último en llegar y si hay varios, se aplica el criterio del id. Además, cada bicicleta muestra los identificadores de los docks en los que ha cargado.

```
( Final information )
(-------------------)
(-------------------)
( Electric Bikes )
(-------------------)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC00_003)
(Dock: CC01_003)
(Dock: CC02_003)
(ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 48, 10-11, 1-1)
(Dock: CC01_003)
```

A continuación se muestran las estaciones de bicicletas ordenadas por el número de bicicletas cargados en ellas (y en caso de empate, por su id). Para cada estación de bicicletas, se muestran sus docks ordenados por el criterio en el que están almacenados en la estación de bicicletas (el mismo definido en el apartado de estaciones de bicicletas) y para cada dock se muestran las bicicletas cargadas en el mismo (que están ordenadas cronológicamente según el momento en el que se han cargado, es decir, la primera en cargar aparece primera).

```
( Bike Stations )
(-------------------)
(BikeStation: CC01, 2, 10-11)
(Dock: CC01_003, 80wh, 0.8€, 2, 32.0€)
(ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 48, 10-11, 1-1)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC01_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC01_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC01_000, 20wh, 0.2€, 0, 0.0€)
```

4. FICHERO COMPLETO.

Por último, se muestra el fichero de ejemplo completo:

```
( Compañía EBBike Cáceres )
(-------------------)
( Electric Bikes )
(-------------------)
(ElectricBike: EBCC0, 15wh, 15wh, 0, 0.0€, 0, 10-13, 10-11, 1-1)
(ElectricBike: EBCC1, 30wh, 30wh, 0, 0.0€, 0, 8-4, 10-5, 19-19)
(-------------------)
( Bike Stations )
(-------------------)
(BikeStation: CC00, 0, 10-5)
(Dock: CC00_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC00_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC00_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC00_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC01, 0, 10-11)
(Dock: CC01_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC01_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC01_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC01_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC02, 0, 14-16)
(Dock: CC02_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC02_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC02_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC02_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC03, 0, 8-4)
(Dock: CC03_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC03_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC03_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC03_000, 20wh, 0.2€, 0, 0.0€)
(-------------------)
( Simulation start )
(-------------------)
(step: 0 - ElectricBike: EBCC0, 15wh, 10wh, 0, 0.0€, 0, 10-12, 10-11, 1-1)
(step: 0 - ElectricBike: EBCC1, 30wh, 25wh, 0, 0.0€, 0, 9-5, 10-5, 19-19)
(step: 1 - ElectricBike: EBCC0 recharges: 10wh at dock: CC01_003 with cost: 8.0€
********)
(step: 1 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 0, 10-11, 1-1)
(step: 1 - ElectricBike: EBCC1 recharges: 10wh at dock: CC00_003 with cost: 8.0€
********)
(step: 1 - ElectricBike: EBCC1, 30wh, 30wh, 1, 8.0€, 0, 10-5, 10-11, 19-19)
(step: 2 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 1, 10-11, 1-1)
(step: 2 - ElectricBike: EBCC1, 30wh, 25wh, 1, 8.0€, 0, 10-6, 10-11, 19-19)
(step: 3 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 2, 10-11, 1-1)
(step: 3 - ElectricBike: EBCC1, 30wh, 20wh, 1, 8.0€, 0, 10-7, 10-11, 19-19)
(step: 4 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 3, 10-11, 1-1)
```

```
(step: 4 - ElectricBike: EBCC1, 30wh, 15wh, 1, 8.0€, 0, 10-8, 10-11, 19-19)
(step: 5 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 4, 10-11, 1-1)
(step: 5 - ElectricBike: EBCC1, 30wh, 10wh, 1, 8.0€, 0, 10-9, 10-11, 19-19)
(step: 6 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 5, 10-11, 1-1)
(step: 6 - ElectricBike: EBCC1, 30wh, 5wh, 1, 8.0€, 0, 10-10, 10-11, 19-19)
(step: 7 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 6, 10-11, 1-1)
(step: 7 - ElectricBike: EBCC1 recharges: 30wh at dock: CC01_003 with cost: 24.0€
********)
(step: 7 - ElectricBike: EBCC1, 30wh, 30wh, 2, 32.0€, 0, 10-11, 14-16, 19-19)
(step: 8 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 7, 10-11, 1-1)
(step: 8 - ElectricBike: EBCC1, 30wh, 25wh, 2, 32.0€, 0, 11-12, 14-16, 19-19)
(step: 9 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 8, 10-11, 1-1)
(step: 9 - ElectricBike: EBCC1, 30wh, 20wh, 2, 32.0€, 0, 12-13, 14-16, 19-19)
(step: 10 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 9, 10-11, 1-1)
(step: 10 - ElectricBike: EBCC1, 30wh, 15wh, 2, 32.0€, 0, 13-14, 14-16, 19-19)
(step: 11 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 10, 10-11, 1-1)
(step: 11 - ElectricBike: EBCC1, 30wh, 10wh, 2, 32.0€, 0, 14-15, 14-16, 19-19)
(step: 12 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 11, 10-11, 1-1)
(step: 12 - ElectricBike: EBCC1 recharges: 25wh at dock: CC02_003 with cost: 20.0€
********)
(step: 12 - ElectricBike: EBCC1, 30wh, 30wh, 3, 52.0€, 0, 14-16, 19-19)
(step: 13 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 12, 10-11, 1-1)
(step: 13 - ElectricBike: EBCC1, 30wh, 25wh, 3, 52.0€, 0, 15-17, 19-19)
(step: 14 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 13, 10-11, 1-1)
(step: 14 - ElectricBike: EBCC1, 30wh, 20wh, 3, 52.0€, 0, 16-18, 19-19)
(step: 15 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 14, 10-11, 1-1)
(step: 15 - ElectricBike: EBCC1, 30wh, 15wh, 3, 52.0€, 0, 17-19, 19-19)
(step: 16 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 15, 10-11, 1-1)
(step: 16 - ElectricBike: EBCC1, 30wh, 10wh, 3, 52.0€, 0, 18-19, 19-19)
(step: 17 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 16, 10-11, 1-1)
(step: 17 - ElectricBike: EBCC1 at target destination ********)
(step: 17 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 0, 19-19, 19-19)
(step: 18 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 17, 10-11, 1-1)
(step: 18 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 1, 19-19, 19-19)
(step: 19 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 18, 10-11, 1-1)
(step: 19 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 2, 19-19, 19-19)
(step: 20 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 19, 10-11, 1-1)
(step: 20 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 3, 19-19, 19-19)
(step: 21 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 20, 10-11, 1-1)
(step: 21 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 4, 19-19, 19-19)
(step: 22 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 21, 10-11, 1-1)
(step: 22 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 5, 19-19, 19-19)
(step: 23 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 22, 10-11, 1-1)
(step: 23 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 6, 19-19, 19-19)
(step: 24 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 23, 10-11, 1-1)
(step: 24 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 7, 19-19, 19-19)
(step: 25 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 24, 10-11, 1-1)
(step: 25 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 8, 19-19, 19-19)
(step: 26 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 25, 10-11, 1-1)
(step: 26 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 9, 19-19, 19-19)
(step: 27 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 26, 10-11, 1-1)
(step: 27 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 10, 19-19, 19-19)
(step: 28 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 27, 10-11, 1-1)
(step: 28 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 11, 19-19, 19-19)
(step: 29 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 28, 10-11, 1-1)
(step: 29 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 12, 19-19, 19-19)
(step: 30 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 29, 10-11, 1-1)
(step: 30 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 13, 19-19, 19-19)
(step: 31 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 30, 10-11, 1-1)
(step: 31 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 14, 19-19, 19-19)
(step: 32 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 31, 10-11, 1-1)
(step: 32 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 15, 19-19, 19-19)
(step: 33 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 32, 10-11, 1-1)
(step: 33 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 16, 19-19, 19-19)
(step: 34 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 33, 10-11, 1-1)
(step: 34 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 17, 19-19, 19-19)
(step: 35 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 34, 10-11, 1-1)
(step: 35 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 18, 19-19, 19-19)
(step: 36 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 35, 10-11, 1-1)
(step: 36 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 19, 19-19, 19-19)
(step: 37 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 36, 10-11, 1-1)
```

```
(step: 37 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 20, 19-19, 19-19)
(step: 38 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 37, 10-11, 1-1)
(step: 38 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 21, 19-19, 19-19)
(step: 39 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 38, 10-11, 1-1)
(step: 39 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 22, 19-19, 19-19)
(step: 40 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 39, 10-11, 1-1)
(step: 40 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 23, 19-19, 19-19)
(step: 41 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 40, 10-11, 1-1)
(step: 41 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 24, 19-19, 19-19)
(step: 42 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 41, 10-11, 1-1)
(step: 42 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 25, 19-19, 19-19)
(step: 43 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 42, 10-11, 1-1)
(step: 43 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 26, 19-19, 19-19)
(step: 44 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 43, 10-11, 1-1)
(step: 44 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 27, 19-19, 19-19)
(step: 45 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 44, 10-11, 1-1)
(step: 45 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 28, 19-19, 19-19)
(step: 46 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 45, 10-11, 1-1)
(step: 46 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 29, 19-19, 19-19)
(step: 47 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 46, 10-11, 1-1)
(step: 47 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 30, 19-19, 19-19)
(step: 48 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 47, 10-11, 1-1)
(step: 48 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 31, 19-19, 19-19)
(step: 49 - ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 48, 10-11, 1-1)
(step: 49 - ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(--------------------)
( Final information )
(--------------------)
(--------------------)
( Electric Bikes )
(--------------------)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC00_003)
(Dock: CC01_003)
(Dock: CC02_003)
(ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 48, 10-11, 1-1)
(Dock: CC01_003)
(--------------------)
( Bike Stations )
(--------------------)
(BikeStation: CC01, 2, 10-11)
(Dock: CC01_003, 80wh, 0.8€, 2, 32.0€)
(ElectricBike: EBCC0, 15wh, 15wh, 1, 8.0€, 48, 10-11, 1-1)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC01_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC01_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC01_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC00, 1, 10-5)
(Dock: CC00_003, 80wh, 0.8€, 1, 8.0€)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC00_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC00_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC00_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC02, 1, 14-16)
(Dock: CC02_003, 80wh, 0.8€, 1, 20.0€)
(ElectricBike: EBCC1, 30wh, 5wh, 3, 52.0€, 32, 19-19, 19-19)
(Dock: CC02_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC02_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC02_000, 20wh, 0.2€, 0, 0.0€)
(BikeStation: CC03, 0, 8-4)
(Dock: CC03_003, 80wh, 0.8€, 0, 0.0€)
(Dock: CC03_002, 60wh, 0.6€, 0, 0.0€)
(Dock: CC03_001, 40wh, 0.4€, 0, 0.0€)
(Dock: CC03_000, 20wh, 0.2€, 0, 0.0€)
```